Git and GitHub – Notes

1. Introduction to Version Control

What is Version Control?

Version Control is a system that records changes to files over time so that specific versions can be recalled later.

It is mainly used in software development to manage source code changes.

Why Version Control is Important

• Tracks history of changes

• Allows multiple developers to work together

• Helps revert to previous versions

• Maintains project backup

• Supports automation and CI/CD pipelines

Version control is a core component of DevOps workflows.

2. What is Git?

Git is a distributed version control system (DVCS) used to track changes in source code during software development.

Git was created by Linus Torvalds in 2005 for Linux kernel development.

Features of Git

• Distributed architecture

• High performance

• Efficient branching and merging

• Secure and reliable

• Works offline

• Supports collaboration

3. What is GitHub?

GitHub is a cloud platform that hosts Git repositories.

It provides tools for:

• Code storage

• Collaboration

• Version tracking

• Code review

• Project management

GitHub Features

• Remote repositories

• Pull requests

• Issue tracking

• GitHub Actions (CI/CD)

• Code collaboration

4. Git Architecture

Git works in three main areas:

1. Working Directory

The directory where files are created, edited, and modified.

2. Staging Area (Index)

Area where files are prepared before committing.

3. Repository

The place where committed files are stored permanently.

Git Workflow

Working Directory

↓

Staging Area

↓

Local Repository

↓

Remote Repository (GitHub)

5. Basic Git Commands

These commands are used to manage repositories.

Repository Commands

Command Description

git init Initialize new Git repository

git clone <repo_url> Copy repository from remote server

git status Show file changes

git add <file> Add file to staging area

git commit -m "message" Save changes with message

git push Upload changes to remote repository

git pull Download and merge updates

git fetch Download changes without merging

git log Show commit history

git diff Show differences between versions

These commands form the core Git workflow.

6. Branching in Git

What is a Branch?

A branch is a separate line of development.

It allows developers to work on features independently without affecting the main project.

Branch Commands

Command Purpose

git branch List branches

git branch <name> Create new branch

git checkout <branch> Switch branch

git checkout -b <branch> Create and switch branch

git merge <branch> Merge branch into current branch

git branch -d Delete branch

Branches allow parallel development.

7. Remote Repository Commands

Remote repositories are stored on servers like GitHub.

Command Description

git remote -v Show remote repository URLs

git remote add Add remote repository

git remote remove Remove remote

git push origin <branch> Push branch to remote

git pull origin <branch> Pull changes

git fetch origin Download changes

8. Staging and Commit Management

Git allows developers to manage commits efficiently.

Command Description

git add . Stage all files

git add -A Stage all changes

git reset Remove file from staging

git commit --amend Modify last commit

git commit -a Stage and commit automatically

9. Viewing and Comparing Changes

Git provides commands to analyze repository history.

Command Function

git diff Show file changes

git diff --staged Show staged changes

git log View commit history

git log --oneline Short commit history

git log --graph Show graphical history

git blame <file> Show who edited each line

10. Undoing Changes

Sometimes developers need to undo changes.

Command Description

git reset Move repository to previous commit

git reset --hard Delete changes permanently

git reset --soft Reset commit but keep changes

git revert Undo commit safely

git restore Restore files

These commands help recover or revert changes.

11. Git Stashing

Stashing temporarily saves changes that are not ready to be committed.

Command Description

git stash Save changes temporarily

git stash list Show stash list

git stash pop Apply and remove stash

git stash apply Apply stash without removing

12. Git Tags

Tags are used to mark important commits (usually releases).

Command Description

git tag List tags

git tag <name> Create tag

git tag -a Annotated tag

git push origin <tag> Push tag

git push origin --tags Push all tags

13. Git Hooks

Git hooks are scripts that run automatically during certain Git events.

Examples:

Hook Purpose

pre-commit Run checks before commit

post-commit Run tasks after commit

pre-push Validate before pushing

commit-msg Enforce commit message format

Hooks help automate tasks.

14. Git Submodules

Submodules allow external repositories to be included inside a project.

Commands:

Command Purpose

git submodule add <repo_url> Add submodule

git submodule init Initialize submodules

git submodule update Update submodules

Submodules help manage dependencies.

15. Git Merge and Rebase

Merge

Merge combines changes from one branch into another.

Example:

git merge feature-branch

Rebase

Rebase moves commits from one branch to another.

Example:

git rebase main Rebase creates a cleaner commit history.

16. Merge Conflicts

Merge conflicts occur when two developers modify the same part of a file.

Git cannot automatically merge those changes.

Steps to Resolve Conflicts

1. Run git status

2. Identify conflicting files

3. Edit and resolve conflicts

4. Add resolved files

5. Commit changes

17. Pull Request (PR)

A Pull Request is a feature used in GitHub.

It allows developers to request merging their branch into another branch.

Benefits

• Code review

• Team collaboration

• Quality control

18. Git in DevOps

Git plays a major role in DevOps workflows.

CI/CD Integration

Common commands used:

• git push

• git fetch --all

• git merge

• git tag

These commands trigger automated build and deployment pipelines.

19. Infrastructure as Code (IaC)

Git repositories store infrastructure configuration files.

Examples:

• Terraform

• Ansible

• Kubernetes

Developers can manage infrastructure just like code.

20. GitOps

GitOps is a DevOps practice where Git repositories act as the single source of truth for deployments.

Workflow:

Developer pushes code

↓

CI/CD pipeline triggered

↓

Application deployed automatically

21. Git Rollback

If deployment fails, developers can revert to a previous version.

Commands used:

git revert <commit_id>

git reset --hard <commit_id>

git checkout <commit_id>

Rollback ensures system stability.

22. Git for Team Collaboration

Git helps teams collaborate efficiently.

Common workflow:

1. Create feature branch

2. Develop code

3. Commit changes

4. Push branch

5. Create Pull Request

6. Review code

7. Merge branch

Important Commands

git init

git clone

git add

git commit

git push

git pull

git fetch

git branch

git checkout

git merge

git log

git diff

git stash

git tag

git reset

git rever